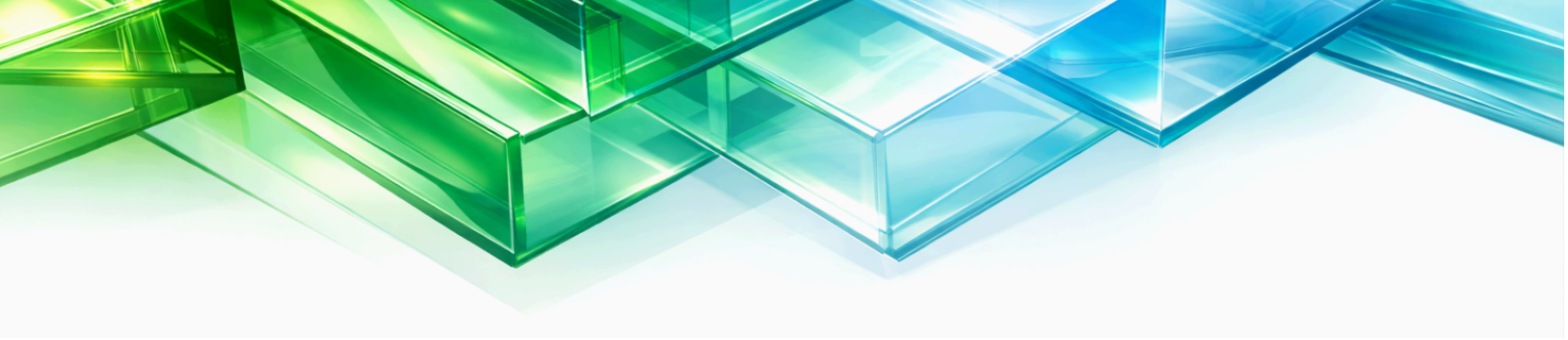

Глоссарий — базовые понятия для RAG

1. **RAG (Retrieval-Augmented Generation)** — комбинация Retriever (извлечение релевантных документов) + Generator (LLM), где генерация подкрепляется найденной информацией. **Важно**: уменьшает галлюцинации и помогает LLM давать фактические ответы.

2. **Retrieval (извлечение)** — процесс поиска и выбора документов/фрагментов, релевантных запросу пользователя. **Важно**: от качества retrieval напрямую зависит точность итогового ответа.

3. **Generation (генерация)** — создание текстового ответа моделью (LLM) на основе входа и извлеченных документов. **Важно**: финальная «лицевая» часть RAG, отвечает за формулировку ответа.

4. **LLM (Large Language Model)** — большая языковая модель, генерирует текст и может использовать извлеченные данные. **Важно**: основной генератор в RAG; его поведение и размер влияют на качество ответов.

5. **Эмбеддинг (embedding)** — векторное представление текста (слов, предложений, документов), сохраняющее семантику. **Важно**: лежит в основе семантического (векторного) поиска.

6. **Эмбеддер (embedder)** — модель или функция, которая преобразует текст в эмбеддинги. **Важно**: выбор эмбеддера влияет на качество семантического соответствия.

7. **Retriever (ретривер)** — компонент, который выполняет поиск по индексу и возвращает кандидаты документов. **Важно**: первый фильтр, от которого зависит покрытие релевантных фрагментов.

8. **Re-ranker (реранкер)** — модель, которая переставляет в порядке релевантности предварительно найденные кандидаты. **Важно**: повышает качество топ-k результатов перед передачей в LLM.

9. **Sparse retrieval (разреженный поиск)** — поиск на основе лексико-статистических представлений (TF-IDF, BM25). **Важно**: эффективен для точного совпадения ключевых слов и коротких запросов.

10. **Dense retrieval (плотный / векторный поиск)** — поиск по эмбеддингам (семантическое сравнение векторов). **Важно**: лучше работает с перефразировками и синонимами.

11. **BM25** — классическая лексико-статистическая функция ранжирования (улучшенная TF-IDF). **Важно**: быстрый, интерпретируемый, часто используется в гибридных системах.

12. **TF-IDF** — мера важности слова в документе относительно корпуса (term frequency–inverse document frequency). **Важно**: базовый способ представления документов в sparse retrieval.

13. **Hybrid search (гибридный поиск)** — комбинация sparse (BM25) и dense (векторного) поиска. **Важно**: объединяет преимущества точного совпадения и семантики.

14. **Vector store (векторное хранилище / индекс)** — база для хранения эмбеддингов и выполнения поиска ближайших соседей. **Важно**: инфраструктурный компонент для dense retrieval.

15. **ANN (Approximate Nearest Neighbors)** — алгоритмы приближенного поиска ближайших векторов (FAISS, HNSW). **Важно**: масштабируемость и скорость при миллионах векторов.

16. **FAISS / HNSW / Annoy** — популярные библиотеки/алгоритмы для ANN-индексов. **Важно**: выбор влияет на скорость, память и точность поиска.

17. **Cosine similarity (косинусная схожесть)** — мера углового сходства между векторами; часто используется для оценки близости эмбеддингов. **Важно**: стандартная мера для семантического соответствия.

18. **Dot product (скалярное произведение)** — мера, часто использующаяся как альтернатива косинусу (в зависимости от нормализации). **Важно**: быстрее в вычислении при некоторых индексах; влияет на ранжирование.

19. **Embedding dimension (размерность эмбеддинга)** — число координат в векторе эмбеддинга (например, 768, 1024). **Важно**: баланс между выразительностью и затратами по памяти/вычислениям.

20. **Chunking (чанкинг)** — разбиение документов на фрагменты (чанки) для индексирования и извлечения. **Важно**: влияет на гранулярность поиска и пригодность контекста для LLM.

21. **Context window (окно контекста)** — максимальное число токенов/символов, которые LLM может обработать за раз. **Важно**: определяет, сколько информации можно подать модели одновременно.

22. **Prompt engineering (инженерия подсказок)** — практика формулирования входа (промпта) для LLM, включая инструкцию + retrieved docs. **Важно**: существенно влияет на корректность и полезность генерации.

23. **Multiquery / Multi-query retrieval** — генерация нескольких перефразированных или специализированных запросов для повышения покрытия поиска. **Важно**: повышает шанс найти релевантные фрагменты при разных формулировках.

24. **Recall@k** — доля релевантных документов, найденных в топ-k результатов. **Важно**: ключевая метрика для измерения покрытия retrieval.

25. **Precision@k** — доля найденных в топ-k документов, которые релевантны. **Важно**: показывает чистоту возвращаемых результатов.

26. **MRR (Mean Reciprocal Rank)** — среднее обратное значение позиции первого релевантного документа. **Важно**: оценивает ранжирование с акцентом на первую релевантную позицию.

27. **MAP (Mean Average Precision)** — средняя по запросам средняя точность ранжирования (учитывает порядок и позиции релевантов). **Важно**: комплексная метрика качества ранжирования.

28. **NDCG (Normalized Discounted Cumulative Gain)** — метрика ранжирования, учитывающая релевантность и ее убывающую важность с позиции. **Важно**: полезна при многозначной релевантности (graded relevance).

29. **LangChain —** фреймворк/библиотека для построения цепочек работы с LLM, retrieval, инструментами и интеграциями. **Важно**: часто используется для реализации RAG-пайплайнов в продакшене и прототипах.

30. **Hallucination (галлюцинация)** — ситуация, когда LLM выдает неправдивую или несуществующую информацию как факт. **Важно**: одна из главных проблем, которую RAG пытается снизить.

31. **Fine-tuning (дообучение)** — подстройка модели на специфичных данных для улучшения ее поведения. **Важно**: можно улучшить эмбеддеры, реранкеры или LLM под задачу/домены.

32. **Contrastive learning (контрастивное обучение)** — метод обучения эмбеддеров, где позитивные пары сближают, а негативные пары раздвигают вектора. **Важно**: дает сильные семантические представления для dense retrieval.

33. **Latency / Throughput** — задержка ответа; Throughput — пропускная способность (запросы/с). **Важно**: операционные характеристики RAG-системы (важны в продакшене).

34. **Top-k** — количество верхних результатов, которые возвращает retriever (например, top-5, top-10). **Важно**: компромисс между покрытием релевантных документов и нагрузкой на реранкер/LLM.

35. **Токен (token) —** короткая единица текста, на которую разбивает вход модель/токенизатор — может быть символ, subword (часть слова) или слово в зависимости от схемы токенизации.

Важно: размер/количество токенов напрямую влияет на длину промпта, стоимость и на то, сколько контекста поместится в окно модели (context window). Неправильное представление токенов может исказить семантику при эмбеддинге и инференсе.

36. **Токенизатор (tokenizer)** — алгоритм/модуль, который разбивает текст на токены и умеет обратно объединять токены в текст. Популярные подходы: byte-level BPE (GPT), WordPiece (BERT), unigram / SentencePiece. Токенизатор также отвечает за сопоставление токенов с id в словаре (vocab).

Важно: эмбеддеры и LLM должны быть согласованы по токенизации (одна и та же схема). Токенизация влияет на chunking документов, подсчет длин промптов, подсчет стоимости и на качество семантических эмбеддингов (особенно при работе с редкими или составными словами).

37. **Инференс модели (inference / model inference)** — процесс применения обученной модели (LLM, реранкер, эмбеддер) к входным данным для получения предсказания/ответа. В инференсе важны: decoding strategy (greedy, beam, sampling, top-k/top-p), batching, кеширование KV-слоев у автогенеративных трансформеров и управление latency/throughput.

Обратите внимание: в RAG инференс LLM выполняется над промптом, который включает retrieved docs → длина промпта (токены) и выбранная стратегия декодирования определяют задержку, стоимость и качество ответа. Эффективный инференс (батчинг, кеширование, резкое сокращение ненужных токенов) позволяет масштабировать RAG в продакшене и снижать задержки при одновременных запросах.